

SIC/XE 組譯器

姓名：黃俊源

學號：01257128

日期：2026-04-24

Contents

1 專案簡介	1
1.1 專案目標	1
1.2 核心特色	1
1.3 開發環境與工具	1
1.4 系統規格與邊界限制	1
1.4.1 系統邊界條件	2
1.4.2 大小寫識別與處理	2
2 系統架構設計	3
2.1 專案目錄結構設計	3
2.2 核心模組與資料結構職責	3
2.2.1 程式進入點與核心控制	3
2.2.2 詞法分析與語法解析	3
2.2.3 查表與演算法優化	4
2.3 組譯流程演算法 (Two-Pass Logic)	4
3 程式執行與操作說明	5
3.1 專案建置與編譯	5
3.2 執行方式與命令列參數	5
3.2.1 基本語法	5
3.2.2 可用選項	5
3.3 執行範例	6
4 核心技術與實作細節	7
4.1 有限狀態機詞法分析	7

4.1.1	SIC/XE 組合語言的語法規則	7
4.1.2	語法結構說明	8
4.1.3	FSM 狀態轉移與 Token 判定邏輯	8
4.1.4	FSM 狀態轉移設計	9
4.2	指令集與符號表管理等資料結構優化	9
4.2.1	OPTAB 的二分搜尋優化	11
4.2.2	基於動態陣列的指令列表	11
4.2.3	符號表的高效雜湊表實作	11
4.2.4	定址模式與位移計算	11
4.3	目的碼與目的檔生成	12
5	錯誤處理與系統強健性	13
5.1	FSM 語法錯誤捕捉	13
5.2	語意與邏輯錯誤檢查	13
6	結論與心得	14
6.1	專案總結	14
6.2	開發心得與架構反思	14
6.2.1	學習收穫與軟體工程價值	15
A	本作業實現的指令表	16
A.1	指令表總覽	16
A.2	SIC/XE 指令表	16
A.3	實作的假指令	16
B	開發過程中所使用的 AI 工具與對話紀錄	17

1 專案簡介

1.1 專案目標

本專案旨在實作一個基於 C11 標準的 SIC/XE 組合語言組譯器 (Assembler)。其核心功能是將符合 SIC/XE 指令集架構的組合語言原始碼 (`.asm`) 轉換為計算機可執行之目的程式 (`.obj`)。本實作不僅支援標準的 SIC 指令，亦支援 XE 架構的擴充功能，如 Format 4 指令、相對定址模式等。

1.2 核心特色

- 完整指令集支援：支援 Format 1、2、3 及 4（前綴 +）之指令解析。
- 定址模式自動處理：自動計算 PC 相對定址 (PC-Relative) 與 Base 相對定址 (Base-Relative) 之位移量 (Displacement)。
- 嚴謹的語法解析：採用有限狀態機 (FSM) 模型進行詞法分析，確保能精準捕捉 Label、Mnemonic 與 Operands，並具備基礎語法錯誤偵測。
- 跨平台建置：使用 CMake 進行專案自動化建置，支援 Linux、macOS 與 Windows 環境。

1.3 開發環境與工具

- 程式語言：C (C11 標準)。
- 建置系統：CMake 4.1.1。
- 編譯器：GCC / Clang。
- 關鍵資料結構庫：使用 `stb_ds.h` 實作高效能的動態陣列與雜湊表 (Hash Map)。

1.4 系統規格與邊界限制

為確保組譯器在處理各種組合語言檔案時的穩定性，並防範潛在的緩衝區溢位 (Buffer Overflow) 等記憶體安全問題，本專案明確定義了系統的處理邊界，並制定了嚴謹的字元識別規範。

1.4.1 系統邊界條件

各項長度上限皆統一定義於 `constants.h` 中，具體規格如下：

- **單行讀取緩衝區 (`READ_BUFFER_SIZE`)**：配置 1024 Bytes，確保能安全讀取包含極長註解的單行原始碼。
- **標籤長度上限 (`MAX_LABEL_LENGTH`)**：64 字元。
- **助記符長度上限 (`MAX_MNEMONIC_LENGTH`)**：16 字元。
- **運算元長度上限 (`MAX_OPERAND_LENGTH`)**：256 字元，足以應付極長的字串常數宣告 (如 `c'...'`)。
- **T 紀錄長度上限 (`MAX_TRECORD_LENGTH`)**：30 Bytes。此數值嚴格遵循 SIC/XE 目的檔規範，確保單筆 Text Record 的機器碼長度不超過十六進位的 1E。

1.4.2 大小寫識別與處理

在詞法分析與符號建構的過程中，系統針對不同型別的語彙基元 (Token) 採取以下判定策略：

- **標籤與符號 (Label & Symbol)**：視為大小寫敏感 (Case-sensitive)。例如，標籤 `LOOP` 與 `loop` 會被符號表 (SYMTAB) 視為兩個完全不同的獨立參照。
- **助記符與假指令 (Mnemonic & Directives)**：要求嚴格匹配 OPTAB 中定義的大寫字元，以維持組合語言指令結構的統一性。

2 系統架構設計

本專案採用高度模組化的 C 語言專案標準結構，將系統核心邏輯、資料結構定義與測試驗證徹底分離，以確保程式碼的高可讀性與擴展性。

2.1 專案目錄結構設計

為了有效管理大型專案，本組譯器的原始碼與資源依照功能劃分至不同目錄中：

- `src/`：存放所有系統核心運算的 C 語言實作檔（如 `main.c`, `assemble.c`, `parse.c` 等）。
- `include/`：存放所有共用的標頭檔與資料結構定義（如 `instruction.h`, `context.h`, `stb_ds.h` 等），嚴格區分介面與實作。
- `examples/`：存放測試用的 SIC/XE 組合語言原始碼（`.asm`）與預期產出的目的檔（`.obj`），用於驗證系統的正確性。
- `docs/`：存放技術報告的 LaTeX 原始碼與相關圖片資源（如 `fsm.jpg`）。
- `CMakeLists.txt` 與 `build/`：定義跨平台的建置規則，並隔離編譯過程產生的暫存檔與最終可執行檔。

2.2 核心模組與資料結構職責

系統的各個 `.c` 與 `.h` 檔案皆具備單一且明確的職責，模組間的相依關係如下：

2.2.1 程式進入點與核心控制

- `main.c`：系統的進入點。負責解析命令列參數（如選擇標準 SIC 或 SIC/XE 模式），並呼叫組譯主程序。
- `assemble.c/assemble.h`：實作組譯器的核心業務邏輯，精準控制 Pass 1 與 Pass 2 的執行流程，並處理目的檔（Object file）的格式化輸出（包含 H, T, M, E 紀錄）。
- `context.h`：定義組譯過程中的全局狀態（Context），如當前的記憶體位置計數器（LOCCTR）、基址暫存器（BASE）狀態等。

2.2.2 詞法分析與語法解析

- `parse.c/parse.h`：實作基於有限狀態機（FSM）的詞法分析器，負責將單行組合語言原始碼拆解為標籤、助記符與運算元。

- `instruction.h`：定義單行指令的資料結構（`Instruction`），用以封裝解析後的指令內容、控制位元（`n`, `i`, `x`, `b`, `p`, `e`）以及生成的機器碼。

2.2.3 查表與演算法優化

- `opcode.c` / `opcode.h`：封裝 SIC/XE 指令集表（`OPTAB`），並提供基於二分搜尋法（`Binary Search`）的高效 `Opcode` 查找介面。
- `symbol.h`：定義符號表（`SYMTAB`）的結構，負責紀錄程式碼中定義的標籤與其對應的絕對記憶體位址。
- `stb_ds.h`：引入此輕量級的資料結構函式庫，為系統提供高效的「動態陣列（`Dynamic Array`）」以存儲無限量指令，以及「字串雜湊表（`Hash Table`）」以達到 $O(1)$ 的符號表查詢效能。

2.3 組譯流程演算法（Two-Pass Logic）

在模組化架構的基礎上，本系統採用經典的兩次掃描（`Two-Pass`）演算法，以確保所有前向引用（`Forward Reference`）標籤皆能被正確解析。

1. Pass 1（位址分配與符號解析）：

系統透過 `parse.c` 逐行讀取 `.asm` 檔，透過指令長度與假指令（如 `RESW`, `BYTE`）維護 `LOCCTR`，並將所有標籤寫入基於 `Hash Table` 實作的符號表。此階段的結果將依序存入動態陣列中供後續使用。

2. Pass 2（機器碼生成與輸出）：

系統遍歷 `Pass 1` 生成的指令陣列，利用 `opcode.c` 查詢指令對應的 `Opcode`。接著結合符號表查詢目標位址，計算 `PC-Relative` 或 `Base-Relative` 的位移量（`Displacement`），最終組合出十六進位機器碼，並由 `assemble.c` 輸出至 `.obj` 檔案中。

3 程式執行與操作說明

本章節將說明如何使用 CMake 建置本專案，以及編譯完成後，如何透過命令列介面 (CLI) 操作本組譯器並切換不同的執行模式。

3.1 專案建置與編譯

本專案採用跨平台的 CMake 作為自動化建置工具，原始碼遵循 C11 標準。請在專案的根目錄下開啟終端機，並依序執行以下指令進行編譯：

```
$ mkdir build
$ cd build
# 產生建置檔 (可指定編譯模式，此處以 Release 為例)
$ cmake -DCMAKE_BUILD_TYPE=Release ..
# 執行編譯以產生可執行檔
$ make
```

Listing 3.1: 編譯專案指令

編譯成功後，系統將會在 build 目錄下生成名為 main 的可執行檔。

3.2 執行方式與命令列參數

為提升系統的靈活性與使用者體驗，本組譯器設計了完整的命令列參數解析機制，允許使用者靈活指定輸入、輸出檔案，以及切換底層的組譯模式。

3.2.1 基本語法

程式的執行語法如下：

```
$ ./main [options] <inputfile> [outputfile]
```

3.2.2 可用選項

使用者可透過以下參數調整程式行為：

- `-h, --help`：顯示系統幫助訊息與參數說明，並安全退出程式。
- `-v, --version`：顯示組譯器版本資訊與開發者隱藏訊息（彩蛋），並退出程式。
- `-x, --xe`：將系統切換為 SIC/XE 擴充模式進行組譯（此為系統預設模式）。
- `-s, --sic`：將系統切換為標準 SIC 模式進行組譯，此模式下將停用 Format 3/4 等擴充特性。

- `<inputfile>`：必須提供的參數。指定欲組譯的組合語言原始碼檔案路徑（例如 `../examples/textbooksicxe.asm`）。
- `[outputfile]`：選擇性參數。指定輸出的目的檔名稱。若使用者未指定，系統將自動以 `output.obj` 作為預設輸出檔名。

3.3 執行範例

以下提供幾種常見的實務操作情境：

```
# 情境一：以預設的 XE 模式組譯，並明確指定輸出的目的檔名稱
$ ./main -x ../examples/textbooksicxe.asm my_output.obj

# 情境二：以標準 SIC 模式組譯，輸出檔名省略，讓系統使用預設值
$ ./main -s ../examples/standard_sic.asm

# 情境三：查詢系統版本與觸發彩蛋訊息
$ ./main --version
```

Listing 3.2: 執行範例

4 核心技術與實作細節

本章節將深入探討組譯器開發過程中的關鍵技術，包含採用有限狀態機進行語法解析、指令查表的演算法優化，以及相對定址模式下複雜的位移（Displacement）計算邏輯。

4.1 有限狀態機詞法分析

傳統組譯器通常依賴嚴格的欄位格式，例如以行首空白判斷是否存在標籤，且運算元需以逗號緊密分隔（如 `op1,op2`），逗號前後不得包含空白。此類規範雖有助於簡化語法解析，但也降低輸入彈性與使用便利性。為提升系統的寬容度與使用者體驗，本專案於 `parse.c` 中設計並實作更具彈性的有限狀態機（FSM）與 Token 判定機制，使語法解析不再嚴格依賴固定格式，能支援較寬鬆的輸入形式並適應多樣化的撰寫習慣。

4.1.1 SIC/XE 組合語言的語法規則

為了實現上述彈性解析，我們首先定義了 SIC/XE 組合語言之語法結構如下：

```
<statement> ::= [<Label>] [ "+" ] <Mnemonic>
                [ <addr-mode> <Operand1> [ ",", <Operand2> ] ]
                [<Comment>]

<alpha> ::= [a-zA-Z]
<digit> ::= [0-9]
<hexdigit> ::= [a-fA-F0-9]
<alnum> ::= <alpha> | <digit>

<any-char> ::= [\x20-\x7E] // 可見 ASCII 字元

<symbol> ::= <alpha> [ <alpha> | <digit> ]*
<numeric> ::= <digit> <hexdigit>*
<string> ::= "C'" <any-char>* "'" | "X'" (<hexdigit><hexdigit>)* "'"

<Label> ::= <symbol>
<Mnemonic> ::= <alpha>+
<addr-mode> ::= "@" | "#"
<Operand1> ::= <symbol> | <numeric> | <string>
<Operand2> ::= <symbol> | <numeric>

<Comment> ::= "." <any-char>*
```

4.1.2 語法結構說明

- **指令主體 (<statement>)**：一個完整的指令行由標籤、擴充前綴 (+)、助記符、運算元與註解組成。除了 <Mnemonic> 之外，其餘欄位皆為選擇性出現。
- **定址模式 (<addr-mode>)**：透過前綴 @ 標示間接定址，或透過 # 標示立即值定址。若無前綴則預設為簡單定址。
- **運算元型別 (<Operand1> 與 <Operand2>)**：第一運算元支援符號 (Symbol)、數值 (Numeric) 以及用於 BYTE 假指令的字串常數 (C'...' 或 X'...')；若存在第二運算元（通常用於暫存器索引，如 ,X），則僅允許符號或數值。

4.1.3 FSM 狀態轉移與 Token 判定邏輯

- **行首與註解處理**：自動忽略行首空白；若遇到 .（註解）或 EOL（End of Line），則將該行視為註解或空行並直接跳過。
- **智慧標籤識別 (Token 查表)**：擷取第一個 Token 並查詢 OPTAB。若存在於表中，則判定為助記符 (Mnemonic)；否則視為標籤 (Label)。
- **Format 4 指令捕捉**：在解析助記符前先檢查是否有 + 前綴，若存在則標記為 Format 4 擴展格式。
- **定址模式與第一運算元解析 (Operand 1)**：
 - 辨識 #（立即定址）與 @（間接定址）前綴。
 - 支援三種資料型態：符號 (Symbol)、數字 (Numeric)、字串常數 (C'...' 或 X'...')。
 - 數字允許冗餘的前導 0；若以 A-F 開頭，則必須補上前導 0，否則將被視為符號。
- **逗號分隔與第二運算元 (Operand 2)**：
 - 若出現逗號，則必須有第二運算元（僅限符號或數字），否則視為語法錯誤。
 - 若未出現逗號且直接遇到 EOL，則正常結束解析（代表單運算元指令）。
 - 運算元格式需為 op1, op2，且逗號前後可以接受任意數量的空白。
- **嚴謹的錯誤捕捉機制**：所有不符合上述規則的情況（例如字串未閉合、逗號後無內容等）皆會進入 state_error，以確保組譯器穩定運作且不會崩潰。

4.1.4 FSM 狀態轉移設計

解析器會逐字元讀取原始碼，並根據當前字元與狀態進行轉移。整體設計與前述語法規則（彈性空白、運算元解析與錯誤檢查）相互對應，主要狀態如下：

- `state_start`：忽略行首空白；若遇到 `.` 或 `EOL`，則略過該行。
- `state_label_or_mnem`：擷取第一個 Token，並透過查詢 `OPTAB` 進行比對，判定其為 Label 或 Mnemonic。
- `state_wait_mnem`：在 Label 之後過濾多餘空白，並等待 Mnemonic 的出現。若遇到 `EOL`，則表示該行僅有標籤，是錯誤的語法，進入 `state_error`。
- `state_mnem_prefix`：檢查是否存在 `+` 前綴，並進行相應處理。
- `state_mnem`：逐字元收集完整的 Mnemonic。
- `state_wait_op1`：過濾 Mnemonic 與 Operand 1 之間的空白；若遇到 `EOL`，則結束解析。
- `state_op1_prefix`：處理 Operand 1 的前綴字元（如 `#`、`@`）。
- `state_op1` 與 `state_op2`：解析 Operand 1 與 Operand 2 的內容。並且能夠分別符號、數字與字串三種類型。（但 Operand 2 僅允許符號與數字）
- `state_wait_comma`：在 Operand 1 之後過濾空白，並等待逗號分隔符；若遇到 `EOL`，則結束解析。
- `state_wait_op2`：在逗號之後過濾空白，並等待 Operand 2；若遇到 `EOL`，則轉入錯誤狀態。
- `state_done`：表示該行已成功完成解析。
- `state_error`：處理所有不符合語法規則的輸入情況。

詳細的狀態轉移圖示請參見圖 4.1，該圖清晰展示了各狀態之間的轉移邏輯與條件判定，為理解整體解析流程提供了直觀的視覺輔助。

4.2 指令集與符號表管理等資料結構優化

在組譯的過程中，資料結構的選擇與搜尋演算法的效率會直接影響組譯器的整體效能。為此，本系統針對指令集查表、內部指令結構儲存以及符號表管理，皆導入了高效的演算法與動態資料結構設計。

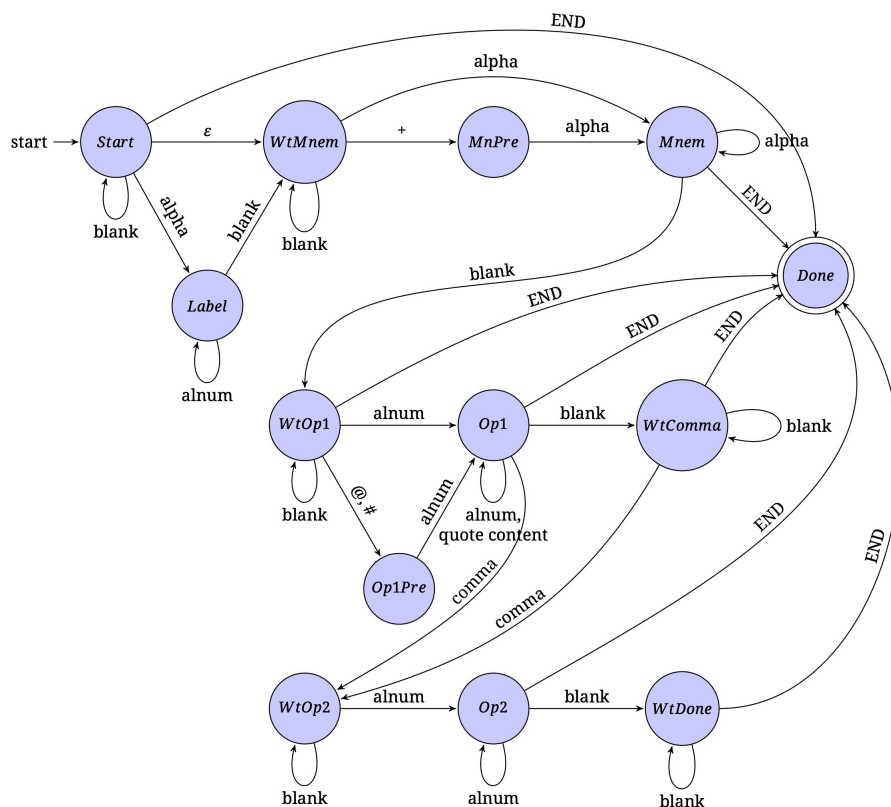


Figure 4.1: 有限狀態機 (FSM) 狀態轉移圖示。

其中 *Wt* 代表等待 (Wait)，*Pre* 代表前綴 (Prefix)，*Mnem*, *Mn* 代表助記符 (Mnemonic)，*Op* 代表運算元 (Operand)。

Alpha 代表字母，alnum 代表字母或數字，blank 代表空白字元 (包含 tab 和空格)，END 代表行尾 (EOL) 或註解。

由於讓狀態轉移更容易讀，我們將開頭到 mnemonic 之間的状态以 NFA 的形式呈現；而不合法的字元則直接轉到 error 狀態，這些轉移在圖中未標示出來。

4.2.1 OPTAB 的二分搜尋優化

SIC/XE 具備豐富的指令集。在 `opcode.c` 中，所有的指令皆預先定義於 OPTAB 靜態陣列（型別為 `OpcodeEntry` 結構）中，並且已經嚴格依照指令助記符（Mnemonic）的英文字母順序排列。

在 Pass 2 轉換目的碼時，若採用傳統的線性搜尋將耗費 $O(n)$ 的時間。因此，本系統在 `searchOpcode` 函式中實作了二分搜尋法（**Binary Search**），將指令比對的時間複雜度大幅降至 $O(\log n)$ ，顯著提升了頻繁查表時的執行效率。

4.2.2 基於動態陣列的指令列表

許多傳統或教學用的組譯器實作，常以固定大小的陣列（Fixed-size array）來存放解析後的指令，這不僅浪費記憶體，更會導致可處理的程式碼行數受限。

為解決此問題，本專案引入了著名的單一標頭檔函式庫 `stb_ds.h`，將儲存解析結果的 `instructions` 變數建構為動態陣列（**Stretchy Buffer**）。當 Pass 1 逐行解析原始碼時，系統會隨需動態分配並擴充陣列容量。此設計徹底解除了檔案大小的硬性上限，使組譯器具備高度的擴展性。

4.2.3 符號表的高效雜湊表實作

符號表（Symbol Table）扮演著串聯 Pass 1 與 Pass 2 的關鍵角色。系統必須在 Pass 1 中頻繁寫入新標籤與位址，並在 Pass 2 遇到相對定址或直接定址時進行大量的位址查詢。

為避免線性搜尋造成的效能瓶頸，本系統同樣利用了 `stb_ds.h` 所提供的字串雜湊表（**String Hash Map**）機制來實作 `symbolTable`。透過 Hash Table，我們能以平均時間複雜度 $O(1)$ 的極高效率完成符號的「重複定義檢查」與「位址查找」，確保組譯器在處理包含大量標籤的大型組合語言專案時，依然能保持優異的運算效能。

4.2.4 定址模式與位移計算

這是在 Pass 2 中最具挑戰性的核心。對於 Format 3 與 Format 4 指令，組譯器必須計算六個控制位元（`n`, `i`, `x`, `b`, `p`, `e`）以及運算元的位移量。

PC-Relative 定址模式

預設情況下，Format 3 指令優先採用 PC 相對定址。設目標位址為 `TA` (Target Address)，當前程式計數器為 `PC`（即下一行指令的起始位址），位移量 `disp` 的計算公式為：

$$disp = TA - PC$$

根據 SIC/XE 架構規範，12 位元的 $disp$ 必須落於以下範圍：

$$-2048 \leq disp \leq 2047$$

若計算出的 $disp$ 符合此區間，則設定 $p=1$, $b=0$ ，並將其轉換為二進位補數 (2's complement) 填入目的碼。

Base-Relative 定址模式

當 $disp$ 超出 PC-Relative 的容許範圍時，系統會自動退格嘗試 Base 相對定址（前提是開發者有使用 BASE 假指令設定 Base 暫存器）。其計算公式為：

$$disp = TA - Base$$

此時位移值為無號整數 (Unsigned)，容許範圍為：

$$0 \leq disp \leq 4095$$

若符合條件，則設定 $b=1$, $p=0$ 。若以上兩者皆不符合，系統將報出「Displacement out of bounds」錯誤，提示使用者應改用 Format 4 指令。

4.3 目的碼與目的檔生成

`assemble.c` 負責將組譯結果格式化輸出。

- **Head Record (H)**：在目的檔的開頭寫入 H 紀錄，包含程式名稱、起始位址與程式長度等基本資訊。
- **Text Record (T)**：程式透過緩衝區收集相鄰的機器碼，當累積長度超過 30 個位元組 ($1E_{16}$) 或遇到記憶體不連續（如 RESW）時，即斷行寫入新的 T 紀錄。
- **Modification Record (M)**：對於採用 Format 4 且非立即數的指令 ($e=1$)，由於其包含絕對記憶體位址，必須在目的檔中生成 M 紀錄，以便後續的載入器 (Loader) 進行重定位 (Relocation)。
- **End Record (E)**：在目的檔的結尾寫入 E 紀錄，指定程式的起始執行位址 (Entry Point)。

5 錯誤處理與系統強健性

一個成熟的系統程式必須具備優良的容錯能力與錯誤提示機制。本專案在多個層面實作了防呆與除錯功能，確保組譯器在遇到不合法的原始碼時，不會發生崩潰 (Crash)，並能給予開發者明確的錯誤定位。

5.1 FSM 語法錯誤捕捉

在 `parse.c` 中，當狀態機遇到不符合語法規範的字元（例如：未閉合的字串引號、逗號後缺乏第二運算元等），狀態會立即轉移至 `state_error`。此機制能有效隔離錯誤的指令行，避免污染後續的指令解析，並透過 `errmsg()` 函式印出具體的錯誤原因與發生的行號。

5.2 語意與邏輯錯誤檢查

除了語法層面，`assemble.c` 亦實作了嚴格的語意檢查：

- **未定義符號 (Undefined Symbol)**：在 Pass 2 階段，若運算元參考了不存在於符號表 (SYMTAB) 中的標籤，系統將會報錯並終止該行機器碼的生成。
- **位移越界 (Displacement Out of Bounds)**：當 Format 3 指令無法使用 PC-Relative (超出 -2048 ~ 2047 範圍) 且無法使用 Base-Relative 時，系統會主動提示開發者將該指令改寫為 Format 4。

6 結論與心得

6.1 專案總結

本專案成功以 C11 標準從零打造了一套功能完整且效能優異的 SIC/XE 組譯器。我們不僅落實了經典的兩次掃描（Two-Pass）演算法，更透過引入二分搜尋法（Binary Search）與動態字串雜湊表（Hash Table）等資料結構，確保了系統在處理大型組合語言專案時的執行效率與擴展性。本系統的架構設計具備高度的模組化特徵，是一次對編譯原理與系統軟體（System Software）開發的成功實踐。

6.2 開發心得與架構反思

回顧開發歷程，本專案最大的技術突破與心得，在於針對語法解析器（`parse.c`）的深度重構。

在開發初期，有限狀態機（FSM）的實作採用了傳統的 `switch-case` 與多層嵌套的 `if-else` 來控制狀態轉移。然而，隨著 SIC/XE 語法規則的擴充（如各種定址前綴與資料型態的判定），主解析迴圈的程式碼迅速膨脹。這不僅導致了極高的循環複雜度（Cyclomatic Complexity），也讓後續的除錯與新功能擴充變得寸步難行。

為了解決此一架構瓶頸，我在專案後期導入了「狀態模式（State Pattern）」的物件導向思維，利用 C 語言的函式指標（Function Pointer）對 FSM 進行了徹底的重構：

```
typedef void* (*StateFunc)(char c, char** l, char** r, Instruction* inst);
```

透過將每一個狀態（如 `state_mnem`, `state_op1`）獨立實作為符合 `StateFunc` 簽章的單一函式，原本臃腫的狀態判斷邏輯被完全去耦合。主迴圈只需透過函式指標不斷呼叫當前狀態，並接收回傳的下一狀態記憶體位址即可進行推進：

```
while (*r != '\0' && current_state != state_error && current_state != state_done) {  
    // 透過函式指標直接跳轉，徹底消滅了龐大的 switch-case 判斷  
    current_state = (StateFunc)current_state(*r, &l, &r, &inst);  
  
    if (current_state != last_state) {  
        last_state = current_state;  
        l = r; // 狀態轉移，更新左邊界  
    }  
    r++; // 推進右邊界  
}
```

Listing 6.1: 基於函式指標的 FSM 主迴圈

6.2.1 學習收穫與軟體工程價值

這次的重構經驗讓我深刻體會到軟體工程中「去耦合 (Decoupling)」與「高內聚 (High Cohesion)」的實際威力。利用函式指標取代條件分支，不僅使程式碼的視覺結構大幅簡化，每一個狀態的邏輯也得以被安全地封裝與隔離。

從底層效能的角度來看，捨棄 `switch-case` 的條件比對，改由函式指標直接跳轉記憶體位址，亦能有效減少編譯器生成的跳躍表 (Jump Table) 開銷，並降低 CPU 在迴圈中分支預測錯誤 (Branch Misprediction) 的成本。這是我在本次系統程式實作中，認為最具價值且最引以為傲的技術成長。

A 本作業實現的指令表

A.1 指令表總覽

Table A.1: SIC/XE 指令表總覽

	x0	x4	x8	xC		x0	x4	x8	xC
0x	LDA	LDX	LDL	STA	8x	STF	STT	COMPF	
1x	STX	STL	ADD	SUB	9x	ADDR	SUBR	MULR	DIVR
2x	MUL	DIV	COMP	TIX	Ax	COMPR	SHIFTL	SHIFTR	RMO
3x	JEQ	JGT	JLT	J	Bx	SVC	CLEAR	TIXR	
4x	AND	OR	JSUB	RSUB	Cx	FLOAT	FIX	NORM	
5x	LDCH	STCH	ADDF	SUBF	Dx	LPS	STI	RD	WD
6x	MULF	DIVF	LDB	LDS	Ex	TD		STSW	SSK
7x	LDF	LDT	STB	STS	Fx	SIO	HIO	TIO	

A.2 SIC/XE 指令表

本組譯器實作支援所有的 SIC/XE 指令表，詳細請見老師提供的 SIC_Inst_Table.pdf

A.3 實作的假指令

在本組譯器的實作中，除了支援標準的 SIC/XE 指令外，我們還實作了一些假指令（Assembler directives）以提升程式撰寫的便利性。這些假指令在組譯過程中會被轉換為一系列的實際機器指令，以達到特定的功能。我們實作的假指令列表如表 A.2 所示，涵蓋了常見的資料定義、空間保留以及程式結構控制等功能。

Table A.2: 實作的假指令表

假指令名稱	功能說明
BASE	設定基底地址，用於相對定址模式的位移計算
NOBASE	取消基底地址設定，回復到預設的絕對定址模式
START	指定程式的起始地址，通常用於程式的入口點
END	標示程式結束，並可指定程式入口點
BYTE	定義一個字節常數，並將其存入記憶體
WORD	定義一個字（通常為 3 字節）常數，並將其存入記憶體
RESB	保留指定數量的字節空間，通常用於宣告變數或緩衝區
RESW	保留指定數量的字空間，通常用於宣告變數或緩衝區

B 開發過程中所使用的 AI 工具與對話紀錄

在本專案的開發過程中，我們使用了對話式人工智慧工具（Google Gemini）作為輔助開發手段，以協助解決實作過程中所遇到的技術問題。其應用範圍涵蓋資料結構設計、語法解析邏輯、有限狀態機（FSM）建構，以及程式碼優化等面向。

AI 工具主要用於提供設計建議、比較不同實作方案，以及協助釐清複雜問題的思路，最終決策與實作仍由開發者自行完成與驗證，以確保系統行為之正確性。

以下列出開發過程中具有代表性的對話紀錄摘要（完整內容請參見 AI_prompt 資料夾中的 PDF 檔案）：

1. SIC Opcode 查表資料結構比較 [SIC-Opcode-查表資料結構比較.pdf](#)
2. SIC/XE Format 2 指令暫存器限制 [SIC-XE Format 2 指令暫存器限制.pdf](#)
3. SIC/XE 指令 C 語言解析器設計 [SIC-XE-指令-C-語言解析器.pdf](#)
4. 狀態機設計與實作優化 [狀態機-枚舉與實作優化.pdf](#)